



INTRODUCCIÓN A INTERNET

Web Design

Carlos Rojas-Sánchez

UNIVERSIDAD DEL MAR

Contenido

1. Perspectiva histórica de Internet
2. Conceptos relacionados con Internet y la Web
3. Arquitectura de dos capas y multicapa
4. Protocolo HTTP
5. Evolución del desarrollo de aplicaciones web
6. Tecnologías del lado del cliente
7. Tecnologías del lado del servidor

Perspectiva histórica de Internet

¿Qué es Internet?

- Internet es una **red de redes**: conecta redes independientes para que sus dispositivos intercambien datos.
- Su desarrollo hizo posible la comunicación entre personas e instituciones ubicadas en distintos lugares.
- Sobre Internet funcionan servicios como el correo electrónico, la mensajería y la World Wide Web.

El problema inicial: conectar computadoras

- En la década de 1960, las computadoras eran grandes, costosas y estaban instaladas en instituciones específicas.
- Investigadores de distintas instituciones necesitaban compartir recursos e información.
- Para lograrlo, se desarrollaron redes capaces de comunicar computadoras ubicadas a distancia.

ARPANET: una red experimental

- ARPANET fue una de las redes precursoras de Internet.
- Comenzó a conectar instituciones de investigación en Estados Unidos a finales de la década de 1960.
- Al terminar 1969, contaba con cuatro computadoras anfitrionas conectadas.

Idea clave

Conectar computadoras abrió la posibilidad de compartir información y recursos entre instituciones.

Conmutación de paquetes

- Para transmitir información, los datos se dividen en unidades pequeñas llamadas **paquetes**.
- Los paquetes viajan por la red y se reúnen en el dispositivo de destino.
- Este enfoque permite compartir la capacidad de la red entre muchas comunicaciones.

Analogía

Es parecido a enviar un documento en varios sobres numerados y reconstruirlo cuando llegan a su destino.

TCP/IP: redes que pueden comunicarse

- Para conectar redes diferentes se necesitaban reglas comunes de comunicación.
- **IP** permite identificar destinos y encaminar paquetes entre redes.
- **TCP** permite establecer una comunicación y entregar los datos en orden.
- En 1983, ARPANET adoptó TCP/IP, un hito fundamental en la evolución de Internet.

Expansión académica y nuevos usos

- Durante la década de 1980 se conectaron más redes de instituciones académicas y de investigación.
- El correo electrónico facilitó la colaboración entre personas ubicadas en distintos lugares.
- Internet dejó de entenderse como una sola red: cada vez más redes podían intercambiar información mediante protocolos comunes.

Internet y la World Wide Web

- Internet ya existía cuando se creó la Web.
- En 1989, Tim Berners-Lee propuso en el CERN un sistema de documentos enlazados accesibles mediante Internet.
- La Web facilitó la consulta de información mediante páginas, enlaces y navegadores.

Distinción importante

Internet es la infraestructura de redes; la **Web** es uno de los servicios que funciona sobre ella.

De la investigación al uso cotidiano

- Durante la década de 1990, Internet llegó a más organizaciones y usuarios.
- Se ampliaron sus usos: búsqueda de información, comunicación, publicación y comercio.
- Después, las conexiones móviles permitieron acceder a estos servicios desde teléfonos y otros dispositivos.

- **Seguridad:** proteger redes, dispositivos y comunicaciones frente a ataques.
- **Privacidad:** cuidar los datos personales de quienes utilizan los servicios.
- **Escalabilidad:** permitir que la infraestructura atienda a un número creciente de usuarios y dispositivos.
- **Neutralidad de la red:** debatir cómo deben tratarse los distintos tipos de tráfico en Internet.

Resumen de la evolución de Internet

1. Se conectaron computadoras para compartir recursos.
2. La conmutación de paquetes permitió transmitir datos.
3. TCP/IP hizo posible la comunicación entre redes.
4. La Web facilitó el acceso a información enlazada.
5. Internet se extendió a la vida cotidiana.

Pregunta para el grupo

¿Por qué no sería correcto decir que Internet nació al mismo tiempo que la World Wide Web?

Conceptos relacionados con Internet y la Web

Proveedor de servicios de Internet (ISP)

- Un **ISP** (*Internet Service Provider*) es una organización que proporciona acceso a Internet.
- Puede ofrecer conexión mediante fibra óptica, cable, redes móviles u otras tecnologías.
- Su función es conectar la red del usuario con otras redes que forman Internet.

Ejemplo

Cuando un estudiante utiliza la conexión de su casa, su proveedor facilita el acceso a sitios y servicios ubicados en otras redes.

- Una **dirección IP** es un identificador utilizado para enviar datos hacia un destino en una red.
- Existen dos versiones principales: **IPv4** e **IPv6**.
- Los dispositivos y servidores utilizan direcciones IP para comunicarse a través de Internet.

Nombres de dominio y DNS

- Un **nombre de dominio** es una dirección fácil de recordar, por ejemplo, **ejemplo.org**.
- El **DNS** (*Domain Name System*) permite encontrar la dirección IP asociada con un dominio.
- Gracias al DNS, no necesitamos memorizar direcciones IP para visitar sitios web.

- **ICANN** es la Corporación de Internet para la Asignación de Nombres y Números.
- Coordina identificadores únicos necesarios para el funcionamiento global de Internet.
- Entre sus funciones se encuentran tareas relacionadas con los nombres de dominio, las direcciones IP y los parámetros de protocolos.

Importante

ICANN no proporciona conexión a Internet: esa es una función de los ISP.

- Un **servicio** es una función que las personas utilizan a través de Internet.
- Un **protocolo** establece reglas para que los sistemas intercambien información.

Ejemplos

- La **Web** es un servicio; **HTTP/HTTPS** son protocolos que utiliza.
- El **correo electrónico** es un servicio; **SMTP** e **IMAP** son algunos de los protocolos relacionados.

- **Internet2** es una iniciativa que reúne instituciones de investigación y educación.
- Proporciona infraestructura y servicios de red para apoyar proyectos académicos y científicos que requieren alta capacidad de comunicación.
- No es una nueva versión de Internet ni debe confundirse con el término «Web 2.0».

Nacimiento de la World Wide Web

- En 1989, Tim Berners-Lee propuso en el CERN un sistema para compartir información mediante documentos enlazados.
- En 1990 desarrolló el primer navegador y servidor web.
- En 1993, el CERN puso el software de la Web en el dominio público, lo que facilitó su expansión.

Estándares fundamentales de la Web

- **URI:** identifica recursos en la Web, como páginas, imágenes y documentos.
- **HTTP:** establece cómo se comunican el navegador y el servidor web.
- **HTML:** define la estructura y el contenido de las páginas web.
- **CSS:** define la presentación visual de las páginas, como colores, fuentes y distribución.

La Web de los años noventa

- Los sitios ofrecían principalmente texto, enlaces e imágenes.
- Muchas páginas eran estáticas: el visitante consultaba información, pero tenía pocas formas de participar.
- Los navegadores gráficos facilitaron el acceso a un público cada vez más amplio.

- Los teléfonos inteligentes permitieron consultar y crear contenido desde casi cualquier lugar.
- Los sitios comenzaron a adaptar su diseño a pantallas de distintos tamaños.
- Las aplicaciones web incorporaron más funciones multimedia e interacción.

La Web actual y sus desafíos

- Hoy usamos la Web para estudiar, trabajar, comprar, comunicarnos y acceder a servicios.
- Las aplicaciones web ofrecen experiencias interactivas y pueden ejecutarse directamente en el navegador.
- Su crecimiento plantea desafíos de privacidad, seguridad, accesibilidad y verificación de información.

Arquitectura de dos capas y multicapa

¿Qué es una capa?

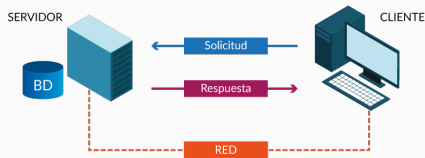
- Una **capa** agrupa responsabilidades relacionadas dentro de una aplicación.
- Por ejemplo, una aplicación necesita:
 - mostrar información al usuario;
 - procesar solicitudes y aplicar reglas;
 - almacenar y recuperar datos.
- Organizar estas responsabilidades en capas permite comprender mejor cómo funciona el sistema.

Idea clave

Una capa describe una responsabilidad; no necesariamente corresponde a una computadora distinta.

Arquitectura de dos capas

- El sistema se organiza en **cliente** y **servidor de datos**.
- El cliente presenta la interfaz y puede ejecutar parte de la lógica de la aplicación.
- El cliente consulta directamente al servidor donde se almacenan los datos.



Ejemplo: sistema de dos capas

Consulta de calificaciones

1. Un usuario abre una aplicación instalada en su computadora.
2. La aplicación solicita las calificaciones directamente a la base de datos.
3. El servidor de datos responde y la aplicación muestra los resultados.

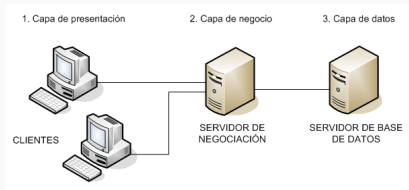
Recorrido

Aplicación cliente → Servidor de base de datos →

Aplicación cliente

Arquitectura de tres capas

1. **Presentación:** muestra la interfaz y recibe las acciones del usuario.
2. **Lógica de negocios:** procesa solicitudes y aplica las reglas del sistema.
3. **Datos:** almacena y recupera la información.



Ejemplo: sistema web de tres capas

Inicio de sesión

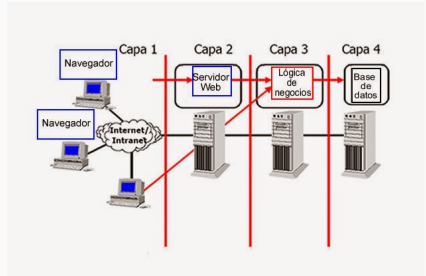
1. El usuario escribe sus datos en el navegador (**presentación**).
2. El servidor recibe la solicitud y comprueba las credenciales (**lógica de negocios**).
3. El servidor consulta la información necesaria (**datos**).
4. El navegador muestra el resultado al usuario.

Recorrido

Navegador → Lógica de negocios → Base de datos

Arquitectura multicapa

- Una aplicación puede dividir sus funciones en más de tres capas.
- Por ejemplo:
presentación, API, lógica de negocio, acceso a datos y almacenamiento.
- Cada capa tiene una responsabilidad definida y se comunica con otras partes del sistema.



¿Qué cambia entre las arquitecturas?

- En **dos capas**, el cliente se comunica directamente con el servidor de datos.
- En **tres capas**, un servidor de aplicación procesa las solicitudes antes de acceder a los datos.
- En una arquitectura **multicapa**, las responsabilidades pueden dividirse aún más.

Actividad breve

En el diagrama de tres capas, identifiquen dónde se realiza cada acción: mostrar un formulario, comprobar una contraseña y consultar las calificaciones.

Protocollo HTTP

¿Qué es HTTP?

- **HTTP** significa *Hypertext Transfer Protocol* (Protocolo de Transferencia de Hipertexto).
- Es un protocolo de aplicación que permite intercambiar recursos y datos en la Web.
- Sigue un modelo de **solicitud y respuesta**: el cliente solicita algo y el servidor responde.

Ejemplo

El navegador solicita una página de productos; el servidor devuelve su contenido.

El ciclo de una solicitud HTTP

1. El usuario abre una dirección o realiza una acción.
2. El navegador envía una solicitud HTTP al servidor.
3. El servidor procesa la solicitud.
4. El servidor devuelve una respuesta HTTP.
5. El navegador muestra el resultado.

Ejemplo

Solicitar **/productos** puede devolver una página HTML o datos en formato JSON.

`https://tienda.ejemplo.org:443/productos?id=15`

- `https`: esquema.
- `tienda.ejemplo.org`: nombre del servidor.
- `443`: puerto; normalmente puede omitirse.
- `/productos`: ruta del recurso.
- `?id=15`: parámetro de consulta.

Puertos habituales

HTTP utiliza normalmente el puerto **80**; HTTPS utiliza normalmente el **443**.

¿Qué contiene una solicitud?

- Un **método** indica la operación solicitada.
- La **ruta** identifica el recurso.
- Las **cabeceras** aportan información adicional.
- Algunas solicitudes incluyen un **cuerpo** con datos para el servidor.

Ejemplo simplificado en HTTP/1.1

GET /productos/15 HTTP/1.1

Host: tienda.ejemplo.org

Accept: application/json

Métodos HTTP más utilizados

GET	Solicita la representación de un recurso.
HEAD	Como GET, pero sin el contenido de la respuesta.
POST	Envía datos para que el servidor los procese.
PUT	Crea o reemplaza el recurso indicado.
PATCH	Solicita una modificación parcial.
DELETE	Solicita eliminar un recurso.

Nota

El servidor determina qué métodos admite para cada recurso.

Ejemplo: enviar datos con POST

- El cliente puede incluir datos en el cuerpo de la solicitud.
- La cabecera **Content-Type** indica el formato de esos datos.

Solicitud simplificada

```
POST /productos HTTP/1.1  
Host: tienda.ejemplo.org  
Content-Type: application/json
```

```
{"nombre": "Cuaderno", "precio": 35}
```

El servidor valida los datos y decide cómo procesarlos.

¿Qué contiene una respuesta?

- Un **código de estado** indica el resultado.
- Las **cabeceras** describen la respuesta.
- El **cuerpo**, cuando existe, contiene HTML, JSON u otros datos.

Respuesta simplificada en HTTP/1.1

HTTP/1.1 200 OK

Content-Type: application/json

```
{"id":15,"nombre":"Cuaderno"}
```

Códigos de estado y HTTPS

- **2xx:** la solicitud se procesó correctamente. Ejemplo: **200 OK**.
- **3xx:** se requiere una acción adicional, como seguir una redirección.
- **4xx:** hay un problema relacionado con la solicitud. Ejemplo: **404 Not Found**.
- **5xx:** el servidor encontró un error. Ejemplo: **500 Internal Server Error**.

HTTP y HTTPS

HTTPS utiliza HTTP sobre una conexión protegida mediante TLS para cifrar la comunicación.

Evolución del desarrollo de aplicaciones web

De páginas informativas a aplicaciones web

- Los primeros sitios públicos presentaban información mediante páginas y enlaces, con pocas opciones de participación.
- Con el tiempo se extendieron los formularios, las cuentas, los comentarios y el contenido creado por los usuarios.
- Actualmente, muchas aplicaciones web permiten trabajar, comunicarse o colaborar directamente desde el navegador.

Web 1.0	Predominan la publicación y la consulta de páginas; la participación suele ser limitada.
Web 2.0	Se popularizan los servicios donde los usuarios crean contenido, interactúan y colaboran.

Importante

Estos términos describen tendencias de uso; no son versiones oficiales del protocolo HTTP ni etapas que se sustituyen por completo.

- Un **CMS** permite crear, organizar y publicar contenidos mediante una interfaz de administración.
- Separa el trabajo editorial de parte de la implementación y facilita actualizar un sitio.
- Un CMS puede servir para un blog, un portal institucional o un sitio informativo; no define por sí solo la Web 2.0.

Interacción sin recargar toda la página

- JavaScript puede solicitar datos al servidor después de que una página se haya cargado.
- El navegador recibe la respuesta y modifica los elementos necesarios mediante el DOM.
- Esta forma de interacción suele asociarse con AJAX; actualmente también se emplean **fetch** y JSON.

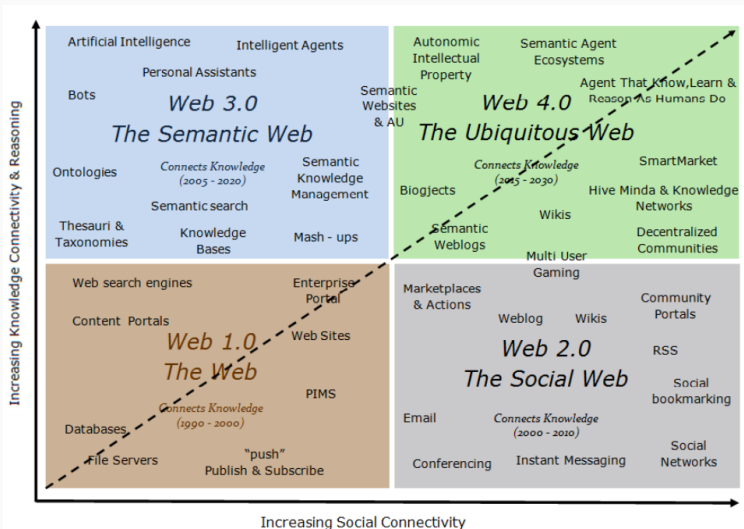
Ejemplo

Al escribir una búsqueda, aparecen resultados nuevos sin cargar de nuevo la página completa.

Datos conectados y acceso desde cualquier lugar

- La **Web semántica** propone describir datos y relaciones con significado que puedan procesar distintos sistemas.
- La idea de una **Web ubicua** destaca el acceso a servicios desde diferentes dispositivos y contextos.
- Ninguna de estas expresiones representa una versión nueva y obligatoria para todos los sitios web.

Evolución de las aplicaciones web



Tecnologías del lado del cliente

¿Qué significa «lado del cliente»?

- El **cliente** es el dispositivo y el programa con el que una persona accede a un sitio web; normalmente, un navegador.
- El navegador recibe recursos de un servidor y los presenta al usuario.
- Las tecnologías del lado del cliente permiten mostrar contenido, definir su apariencia y responder a las interacciones del usuario.

Las tres tecnologías fundamentales

- **HTML** organiza el contenido: títulos, párrafos, imágenes, enlaces y formularios.
- **CSS** controla la presentación: colores, tamaños, espacios y distribución.
- **JavaScript** añade comportamiento: responde a clics, modifica contenido y procesa interacciones.

Idea clave

HTML define *qué hay*; CSS, *cómo se ve*; JavaScript, *cómo se comporta*.

HTML: estructura del contenido

- HTML utiliza **etiquetas** para identificar las partes de una página.
- Una estructura clara ayuda a los navegadores y a las tecnologías de asistencia a interpretar el contenido.

Ejemplo

```
<h1>Mi primera página</h1>  
<p>Bienvenidos a mi sitio web.</p>  
<a href="https://ejemplo.com">Visitar sitio</a>
```

- CSS aplica estilos a los elementos definidos con HTML.
- Permite mantener el contenido separado de su apariencia.
- Una misma regla puede aplicarse a muchos elementos.

Ejemplo

```
h1 {  
    color: navy;  
    text-align: center;  
}
```

JavaScript: comportamiento e interacción

- JavaScript permite responder a acciones del usuario.
- Puede modificar una página sin que sea necesario cargarla completamente de nuevo.

Ejemplo

```
const boton = document.querySelector("#saludar");  
  
boton.addEventListener("click", () => {  
    alert("¡Hola!");  
});
```

El DOM: la página como estructura de objetos

- Al cargar un documento HTML, el navegador crea una representación llamada **DOM** (*Document Object Model*).
- Cada elemento de la página puede identificarse y modificarse mediante JavaScript.
- Por ejemplo, un programa puede cambiar el texto de un título, ocultar una imagen o añadir elementos a una lista.

Formularios e interacción con el usuario

- Los formularios permiten capturar datos mediante campos de texto, botones y listas de selección.
- HTML puede indicar si un campo es obligatorio o qué tipo de dato se espera.
- JavaScript puede revisar los datos y mostrar mensajes antes de enviarlos al servidor.

Atención

La validación en el navegador mejora la experiencia, pero el servidor también debe validar los datos que recibe.

Diseño adaptable o *responsive*

- Una página puede consultarse desde computadoras, tabletas y teléfonos.
- El diseño adaptable reorganiza los elementos según el tamaño de la pantalla.
- CSS ofrece herramientas como *Flexbox*, *Grid* y las consultas de medios (*media queries*).

- El navegador puede solicitar información adicional después de cargar una página.
- JavaScript utiliza herramientas como **fetch** para enviar solicitudes HTTP y recibir respuestas.
- Esto permite, por ejemplo, actualizar una lista de productos sin recargar toda la página.

Ejemplo de recorrido

Usuario hace clic → JavaScript solicita datos → servidor responde → navegador actualiza la página.

Herramientas actuales y cierre

- Las herramientas de desarrollo del navegador permiten inspeccionar HTML, CSS, solicitudes y errores de JavaScript.
- Existen bibliotecas y frameworks para organizar aplicaciones más grandes, como React, Angular y Vue.
- Estas herramientas se apoyan en los fundamentos vistos: **HTML, CSS y JavaScript.**

Pregunta para el grupo

En una página con un botón que cambia de color al pulsarlo, ¿qué función cumple cada una de las tres tecnologías?

Tecnologías del lado del servidor

¿Qué significa «lado del servidor»?

- El **servidor** es el sistema que recibe solicitudes de los clientes y les envía respuestas.
- En él se ejecuta la lógica necesaria para procesar datos, consultar bases de datos y aplicar reglas de negocio.
- El navegador recibe el resultado, pero normalmente no tiene acceso al código interno del servidor.

¿Cómo se comunican cliente y servidor?

1. El navegador envía una **solicitud HTTP**.
2. El servidor identifica qué recurso u operación se solicita.
3. Si es necesario, ejecuta código y consulta una base de datos.
4. Devuelve una **respuesta HTTP** con un resultado.

Ejemplo

Al abrir el perfil de un estudiante, el navegador solicita la información y el servidor devuelve los datos correspondientes.

Solicitudes y respuestas HTTP

- Una solicitud contiene un **método**, una dirección y, en algunos casos, datos adicionales.
- Métodos frecuentes: **GET** para consultar y **POST** para enviar información.
- La respuesta incluye un **código de estado** y puede contener HTML, JSON, imágenes u otros recursos.

Códigos habituales

200: solicitud exitosa **404**: recurso no encontrado

500: error interno del servidor

Lenguajes de programación del servidor

- Se utilizan para implementar operaciones y reglas de una aplicación.
- Algunos ejemplos son **JavaScript** con Node.js, **Python**, **PHP**, **Java**, **C#** y **Ruby**.
- La elección depende de las necesidades del proyecto, el equipo y el entorno donde se ejecutará.

Importante

JavaScript puede ejecutarse tanto en el navegador como en el servidor; el entorno determina a qué recursos puede acceder.

- Una **ruta** relaciona una dirección con una operación de la aplicación.
- El servidor puede usar rutas diferentes para listar, consultar o registrar información.

Ejemplo: sistema de productos

GET /productos: muestra la lista.

GET /productos/15: consulta un producto.

POST /productos: registra un producto.

- Permiten almacenar información que debe permanecer disponible después de cerrar la aplicación.
- El servidor puede consultar, registrar, modificar y eliminar datos según las reglas del sistema.
- Ejemplos: estudiantes, usuarios, productos, pedidos y calificaciones.

Regla básica

El servidor debe comprobar los datos y los permisos antes de realizar cambios en la base de datos.

Autenticación y autorización

- La **autenticación** comprueba la identidad de una persona: «¿quién eres?».
- La **autorización** determina qué operaciones puede realizar: «¿qué tienes permitido hacer?».
- Una sesión o un mecanismo equivalente ayuda a mantener la identidad del usuario entre solicitudes.

Ejemplo

Un estudiante puede consultar sus calificaciones; un docente autorizado puede registrarlas o modificarlas.

- El servidor debe validar los datos recibidos, aunque el navegador ya los haya revisado.
- Debe proteger información sensible y controlar quién puede acceder a cada recurso.
- Las consultas a la base de datos deben implementarse de forma segura para evitar inyecciones.
- Los errores deben comunicarse sin revelar detalles internos del sistema.

- Un servidor puede generar **HTML** con información distinta para cada usuario o solicitud.
- También puede ofrecer una **API** que entregue datos, comúnmente en formato JSON.
- Una aplicación del lado del cliente puede solicitar esos datos y decidir cómo mostrarlos.

Ejemplo

La misma API de productos puede proporcionar información a una página web y a una aplicación móvil.

Herramientas del servidor y cierre

- Los **frameworks** ayudan a organizar rutas, lógica y acceso a datos.
- Ejemplos: Express para Node.js, Django para Python, Laravel para PHP y ASP.NET Core para C#.
- También se utilizan registros de errores, pruebas y herramientas para ejecutar y publicar aplicaciones.

Pregunta para el grupo

Si un estudiante inicia sesión y consulta sus calificaciones, ¿qué tareas realiza el navegador y cuáles el servidor?

Alcances y limitaciones de sitios estáticos

¿Qué es un sitio estático?

- El servidor entrega archivos preparados previamente, como HTML, CSS, JavaScript e imágenes.
- No necesita generar cada página con lógica de servidor propia ni consultar su base de datos en cada solicitud.
- JavaScript puede cambiar lo que ve el usuario o consultar una API externa, aunque los archivos entregados sean estáticos.

- Son adecuados para documentación, portafolios, páginas informativas y materiales de un curso.
- Su alojamiento y distribución suelen ser sencillos.
- Los archivos pueden servirse con rapidez y almacenarse en caché para mejorar el rendimiento.

Limitaciones de los sitios estáticos

- Si cambia el contenido, normalmente hay que editar archivos o generar y publicar una nueva versión.
- Funciones como cuentas de usuario, pagos o guardar datos suelen requerir un servicio de servidor o una API externa.
- Los permisos y datos sensibles deben gestionarse fuera del código que llega al navegador.

Actividad de cierre: elige una arquitectura

1. Un docente publica el temario y las fechas del curso.
2. Un estudiante inicia sesión para ver sus calificaciones.

Preguntas

¿Cuál caso puede resolverse con un sitio estático? ¿En cuál se necesita verificar la identidad y consultar datos protegidos? Expliquen qué hace el cliente y qué hace el servidor.